
Full-Custom Physical Design of an 8×8 6T SRAM Macro in SkyWater Sky130

Engineering Report

Lauai Alerfi

Full-Custom VLSI Layout, Design-Rule Sign-off, and Functional Verification

Process: SkyWater Sky130A (130 nm), open-source PDK (volare)

Tools: Magic 8.3 (layout, DRC, extraction) ngspice (simulation)

Methodology: Human-directed design with AI-assisted implementation

June 2026

Abstract

This report documents the full-custom physical design of an 8×8-bit (64-bit) static RAM macro in the open-source SkyWater Sky130 130 nm process. Beginning from a hand-designed six-transistor bit cell, a complete library of peripheral cells – an inverter, three-input NAND and AND gates, a 3-to-8 row decoder, a precharge/equalize circuit, a write driver, and a cross-coupled sense amplifier – was designed, laid out, and verified. Every cell passes sign-off design-rule checking under the full Sky130 rule deck and is functionally validated by SPICE simulation against its design intent. The bit cell is tiled into a design-rule-clean 8×8 array, and at the macro level the row decoder is placed against the array and its eight word lines are physically routed and verified clean. A data-path periphery column – precharge, write driver, and sense amplifier on one shared bit-line pair – is then built from device primitives, verified clean, and tiled eight-wide; assembled with the array and decoder, the macro performs a correct addressed write and read, returning a written data word bit-exact through the sense amplifiers. The work was carried out with a scripted, reproducible Magic and ngspice flow under human direction, with an AI engineering agent executing the implementation. This document presents the design specification and methodology, each circuit block with its layout and simulation results, the full addressed read/write of the assembled macro, and the key engineering findings – including a calibrated shared-diffusion device-abutment technique and a write-driver write-ability sizing result.

Contents

Abstract	1
1 Executive Summary	3
2 AI-Assisted Design Methodology	3
2.1 Human-defined specification	3
2.2 Root-cause diagnosis of the prior attempt	4
2.3 Agentic implementation	4
2.4 Context	4
3 Objectives and Scope	4
4 Design and Verification Methodology	4
4.1 Programmatic, reproducible layout	4
4.2 Headless device-primitive flow	5
4.3 Shared-diffusion calibration	5
4.4 Verification flow	5
5 Bit Cell and Array	5
5.1 6T bit cell (sram_6t_cell)	5
5.2 Static noise margin	6
5.3 8x8 array (sram_8x8)	6
6 Peripheral Cell Library	8
6.1 NAND3 / AND3	10
6.2 Row decoder (3-to-8)	10

6.3	Precharge / equalize	12
6.4	Write driver	12
6.5	Sense amplifier	13
7	Macro Integration: Address Path	13
7.1	Pitch-translation channel	13
7.2	Result, and a connectivity bug that DRC did not catch	14
8	Data Path: Periphery Column and Full-RAM Operation	14
8.1	Monolithic column primitive	14
8.2	Tiling and the full-macro read/write	14
8.3	Write-ability finding and fix	16
9	Key Engineering Findings	16
10	Results Summary	18
11	Conclusions and Future Work	18

1 Executive Summary

This project delivers a custom-laid-out 8×8-bit (64-bit) 6T SRAM in the Sky130 process, built bottom-up from a hand-designed 6T bit cell through a complete, verified peripheral cell library, and integrated to the point of a physically routed, DRC-clean address path (a 3-to-8 row decoder driving the array’s word lines).

Every cell in the library is sign-off DRC-clean under the full Sky130 rule deck (`drc style drc(full)`, Euclidean) and is functionally verified in ngspice against its design intent. The core storage array (bit cell plus 8×8 tiling) and all six periphery blocks (inverter, NAND3, AND3, 3-to-8 decoder, precharge/equalize, write driver, and sense amplifier) are complete. At the macro level, the decoder is placed against the array and all eight word lines are physically routed and DRC-clean. The data-path periphery is realized as a DRC-clean 12-device column built from device primitives and tiled eight-wide, and the assembled macro (array, periphery, decoder) performs a correct addressed write and read – a data word written to a decoder-selected row is read back bit-exact through the sense amplifiers.

Table 1. Project deliverables and verification status.

Deliverable	Status	Verification
6T bit cell	Complete ✓	DRC 0, device-LVS match, R/W sim
8×8 array	Complete ✓	DRC 0, abutment-connected, GDS
Inverter / NAND3 / AND3	Complete ✓	DRC 0, truth-table sims
3-to-8 row decoder	Complete ✓	DRC 0, one-hot decode sim
Precharge / equalize	Complete ✓	DRC 0, equalize sim
Write driver	Complete ✓	DRC 0, differential-write sim
Sense amplifier	Complete ✓	DRC 0, small-signal resolution sim
Decoder → array WL routing	Complete ✓	DRC 0 (full macro)
Periphery column (<code>pcol_full</code>)	Complete ✓	DRC 0, precharge/write sim
8-column band (<code>pcol_band</code>)	Complete ✓	DRC 0, 96 devices tiled
Full macro addressed read/write	Complete ✓	word 10110010 bit-exact

2 AI-Assisted Design Methodology

This project was conceived and directed by **Lauai Alerfi**. All architectural decisions, design rules, routing strategy, and verification criteria were specified by the author; an AI engineering agent executed and debugged the implementation under that direction.

2.1 Human-defined specification

Before any layout work began, the author defined the complete design specification:

- **Transistor sizing:** PMOS $W = 1.0$, $L = 0.15$; NMOS $W = 0.65$, $L = 0.15$.
- **Cell arrangement:** a left column (PU1 / PD1 / PG1) and a right column (PU2 / PD2 / PG2) for the cross-coupled bit cell.
- **Target netlist, routing rules, and verification requirements** for every block, fixed in advance of layout.

2.2 Root-cause diagnosis of the prior attempt

The author diagnosed the root cause of a previous failed attempt – continuous vertical local-interconnect (li) wires that produced cross-couple shorts in the bit cell – and formulated the corrected routing discipline used throughout this project: the two storage cross-couple links are carried on `metal2` at *different heights* so that they never overlap, and vertical strips are kept on distinct layers per net.

2.3 Agentic implementation

Claude Code (Anthropic) was used as an AI engineering agent running in the Linux terminal via `claude -dangerously-skip-permissions` in the `~/sram_6t/` project directory. From the author’s specifications the agent generated Magic Tcl scripts and ngspice testbenches, maintained a coordinate log in `notes.txt`, and iterated on DRC errors under the author’s direction. The division of labor was deliberate: the author owned the architecture, device sizing, design rules, routing strategy, and pass/fail criteria; the agent owned the mechanical execution – emitting scripts, running `drc check` and `ext2spice`, parsing violations, and proposing coordinate corrections for the author’s review.

2.4 Context

This workflow represents an early real-world application of LLM-based agentic assistance in full-custom VLSI layout, consistent with concurrent academic work on LLM-driven custom layout generation [1] and agentic AI design frameworks for ASIC systems [2]. Total active, human-directed design time was approximately **14–15 hours** from a blank canvas to the macro described in this report.

3 Objectives and Scope

1. A DRC-clean, electrically real 6T SRAM bit cell in Sky130.
2. A tiled 8×8 array with shared word lines (rows) and bit lines (columns).
3. A complete peripheral cell library to make the array addressable and read/write-able: row decoder, precharge/equalize, write driver, sense amplifier, and the standard logic (inverter, NAND3, AND3) the decoder is built from.
4. Top-level macro integration: route the decoder to the array word lines and the periphery to the array bit lines.
5. Full-rule DRC sign-off and functional simulation of every block.

4 Design and Verification Methodology

4.1 Programmatic, reproducible layout

All cells are generated by scripted Magic command files, not interactive drawing. Transistors are instantiated with the Sky130 device generator (which emits DRC-correct `ndiff`, contacts, and wells), then assembled with painted routing, substrate/well taps, and labels. Every build script self-checks `drc(full)` and writes its result, giving a repeatable build → DRC → fix loop.

4.2 Headless device-primitive flow

A key methodology established in this project: the device generator (`gencell`) hangs in headless batch mode, but placing an already-saved cell (`getcell`) works headlessly. The workflow is therefore:

1. **Once**, in an interactive Magic session, `gencell` the needed device primitives and save them as library cells: `dev_nfet_065` ($W = 0.65$), `dev_nfet_100` ($W = 1.0$), `dev_pfet_100` ($W = 1.0$).
2. **All** subsequent cell assembly is pure headless batch (`getcell` + `paint` + `flatten` + `DRC` + `extract`), enabling fast automated iteration.

4.3 Shared-diffusion calibration

Abutting generator devices at the generator’s native pitch (76) leaves a three-unit diffusion gap that cascades into spacing violations (`li.3`, `licon`, `mcon`, `met1`). The fix is shared-diffusion abutment, in which one device’s drain diffusion overlaps the next device’s source. The shared-diffusion pitch was calibrated empirically for the `getcell` placement reference:

- NMOS shared-diffusion pitch = 46λ .
- PMOS shared-diffusion pitch = 44λ .

At these pitches a bare device chain is DRC-clean except for latch-up checks (`LU.2`/`LU.3`), which clear once the cell has its substrate/well ties.

4.4 Verification flow

- **DRC:** the count is correct only after the exact sequence below (the `select top cell` and `catchup` steps are required):

```
drc style drc(full); select top cell; drc check; drc catchup; drc list count total
```

- **Extraction:** `extract all`; `ext2spice` produces a SPICE netlist with named ports (set via Magic `port make`), checked against the reference netlist.
- **LVS:** a device-graph comparison (canonical device signatures (type, gate, {drain, source}, bulk)) against the golden reference. The bit cell, inverter, and precharge – whose nodes are all named – pass formally; gates and drivers with unnamed internal series nodes are signed off by functional simulation instead.
- **Simulation:** ngspice transient against the Sky130 device models (`tt` corner, 1.8V); a DC butterfly sweep for static noise margin.
- **GDS:** every cell and the macro export to GDS (`gds write`) and read back clean.

5 Bit Cell and Array

5.1 6T bit cell (`sram_6t_cell`)

Standard 6T topology: two cross-coupled inverters (`PU1`/`PD1`, `PU2`/`PD2`) plus two access transistors (`PG1`, `PG2`). PMOS $W = 1.0$, NMOS $W = 0.65$, $L = 0.15$.

- **Footprint:** $6.45\mu\text{m} \times 6.01\mu\text{m}$.

- **Routing discipline (avoids cross-couple shorts):** vertical **metal1** for Q/QB and BL/BLB strips; horizontal **metal2** for VDD, GND, and the WL rail; the two cross-couple links run on **metal2** at different heights so they never overlap.
- **Taps:** N-tap per nwell to VDD, P-tap per column to GND, each a full diffusion + local-interconnect + **metal1** contact stack (both the diffusion contact and the li-to-**metal1** contact are required, or the well/substrate floats).
- **Verification:** `drc(full)=0`; extracted netlist is device-isomorphic to the golden reference (ports VDD GND Q QB WL BL BLB); ngspice write-1 / write-0 / read confirm bit-storage and non-destructive read.

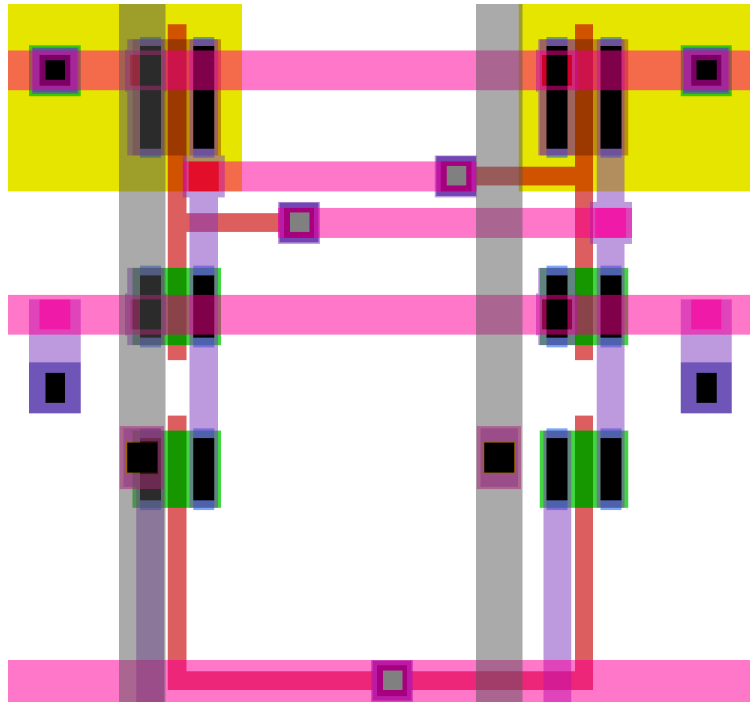


Figure 1. 6T bit-cell layout (Magic, DRC-clean). PMOS pull-ups in the nwell (yellow, top), NMOS below, poly gates (red), **metal1** storage/access nets (purple), **metal2** VDD/GND/WL rails (pink).

5.2 Static noise margin

The cell's stability is quantified by its static noise margin (SNM), the side of the largest square inscribed between the two inverter voltage-transfer curves of the butterfly plot. The curves are obtained by a DC sweep that forces one storage node and measures the other, for two bias conditions: *hold* ($WL=0$) and *read* ($WL=VDD$ with $BL=BLB=VDD$). The extracted values are **hold SNM = 716 mV** and **read SNM = 311 mV**; the read margin is the smaller, as expected, because the access transistors disturb the storage nodes when the word line is asserted.

5.3 8×8 array (`sram_8x8`)

64 instances of the bit cell tiled on a 645λ (x) $\times$ 601λ (y) pitch; word lines and bit lines connect by abutment (WL **metal2** rails span the row width; BL/BLB **metal1** strips span the column height). A left-edge **metal3** VDD strap ties every row's VDD rail via `via2`.

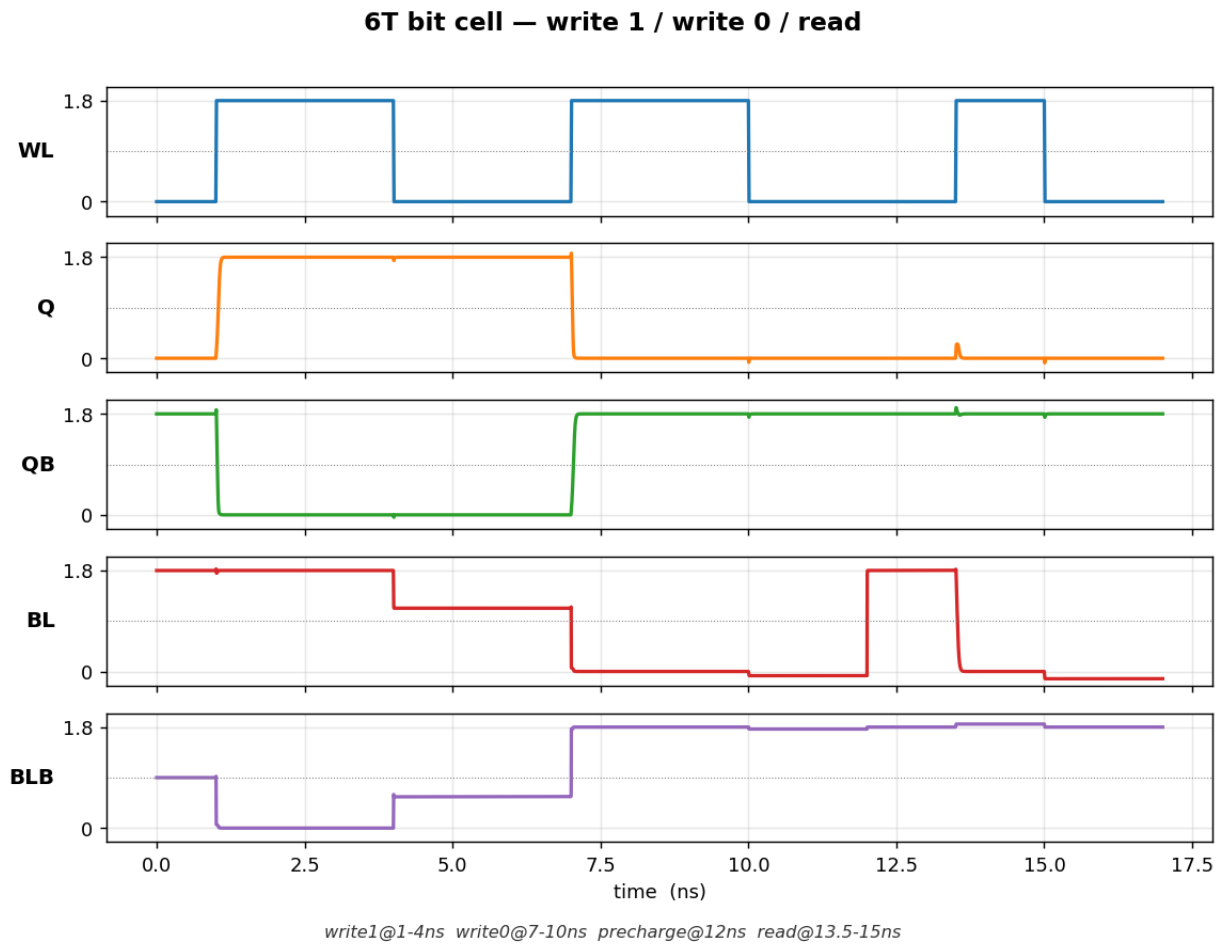


Figure 2. 6T bit-cell read/write transient (ngspice, τ_t , 1.8 V). WL pulses drive write-1 ($Q \rightarrow 1$), write-0 ($Q \rightarrow 0$), and a non-destructive read.

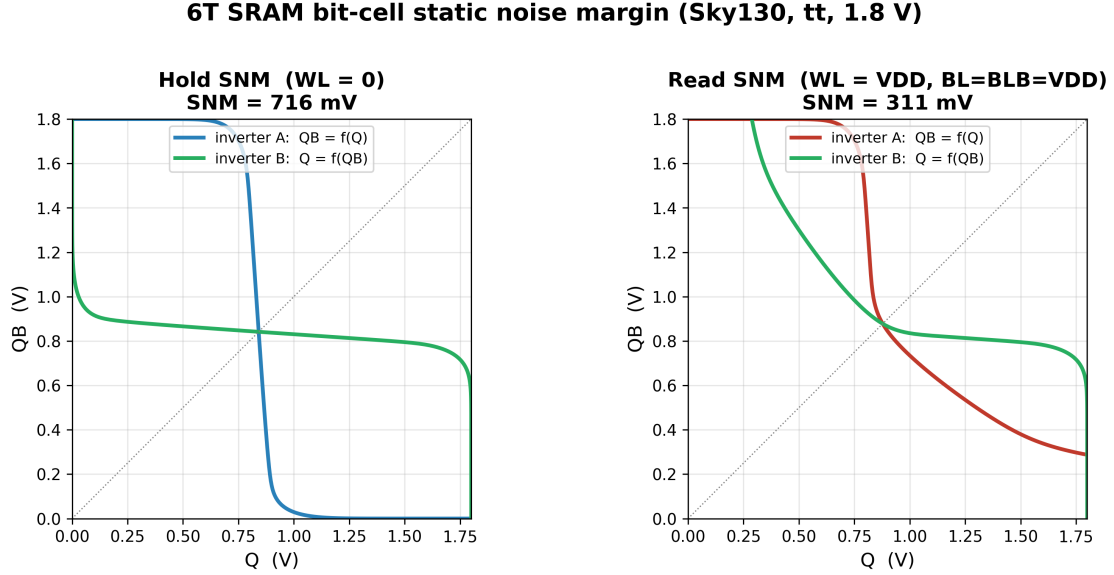


Figure 3. 6T bit-cell static noise margin. Butterfly curves for hold (left) and read (right) bias; the read eye is markedly smaller, giving read SNM = 311 mV vs. hold SNM = 716 mV.

- **Interface:** WL_j (**metal2**) at the left edge, array-internal $y = 18 + 601j$; BL_i (**metal1**) at internal $x = 122 + 645i$, BLB_i at $472 + 645i$ (350 λ apart within a column), exposed at top and bottom edges.
- **Verification:** $drc(full) = 0$; electrically connected; exported to GDS and read back clean.

6 Peripheral Cell Library

All periphery cells were rebuilt on the shared-diffusion flow to reach full-rule DRC sign-off (an earlier generation was functionally correct but only sparse-rule clean; the rebuild closed every full-deck violation). Each re-passes its functional testbench.

Table 2. Peripheral cell library: topology and verification.

Cell	Topology	drc(full)	Functional check
inv	1× PMOS / 1× NMOS, gates tied	0	—
nand3	3 series NMOS + 3 parallel PMOS	0	$Y = \text{NOT}(A \cdot B \cdot C)$ ✓
and3	NAND3 + output inverter	0	$Y = A \cdot B \cdot C$ ✓
decoder	8× AND3 + 6 address rails	0	one-hot 3-to-8 ✓
precharge	3 PMOS (2 pull-up + equalizer)	0	$BL = BLB = 1.8\text{ V}$ ✓
writedrv	4 NMOS, two series pull-down stacks	0	differential write ✓
senseamp_1	cross-coupled latch + tail enable	0	8 mV $\rightarrow$ full rail ✓

The inverter is the elementary standard cell (vertical PMOS-over-NMOS, rails kept clear of the drains, taps with full nwell enclosure):

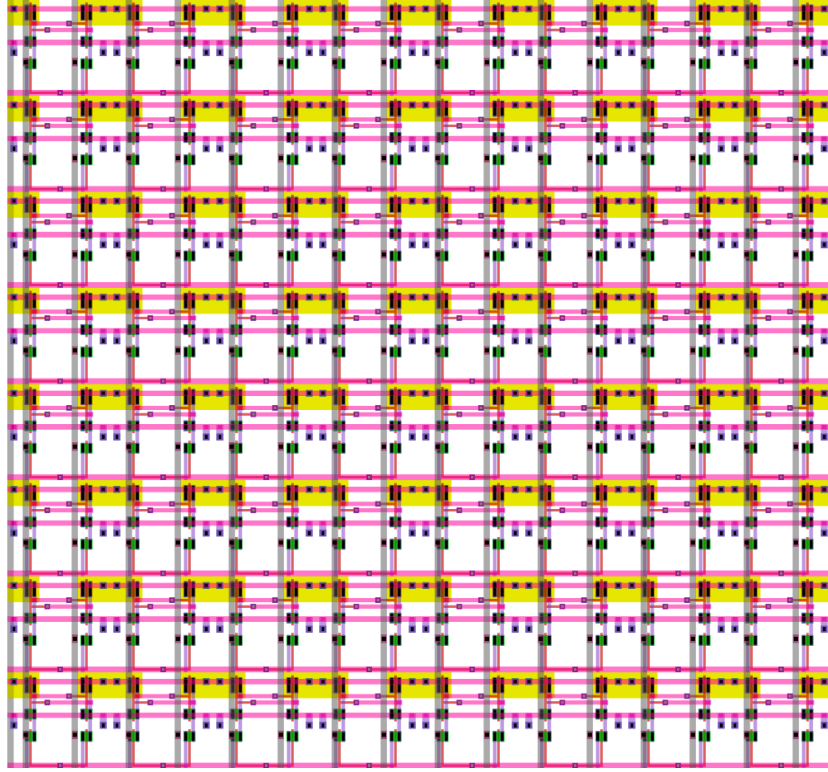


Figure 4. 8×8 array layout (Magic, DRC-clean). 64 bit-cell instances on a $645 \times 601 \lambda$ pitch; word lines and bit lines connect by abutment; vertical `metal13` VDD straps at left.

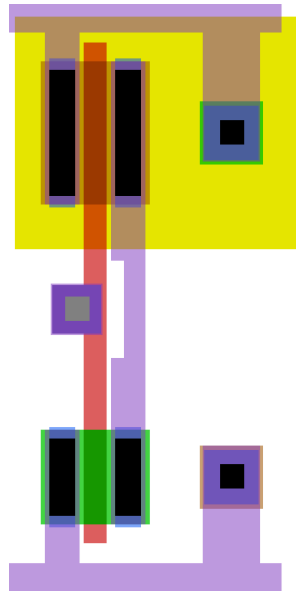


Figure 5. Inverter standard-cell layout (Magic, DRC-clean).

6.1 nand3 / and3

NAND3 is a three-input series NMOS stack (shared-diffusion at pitch 46) with three parallel PMOS whose drain contacts alternate VDD/Y; the Y node bridges over the intervening VDD diffusion on `metal2`. Gate inputs are brought below the cell to `poly`-contact landings, staggered in y so their `metal1` pads meet spacing. AND3 is built monolithically as NAND3 + an output inverter in a single coordinate frame (the NAND output node drives the inverter gate; the inverter drain spine is the AND3 output). Both extract to the exact target netlist with auto-generated ports and pass truth-table simulation.

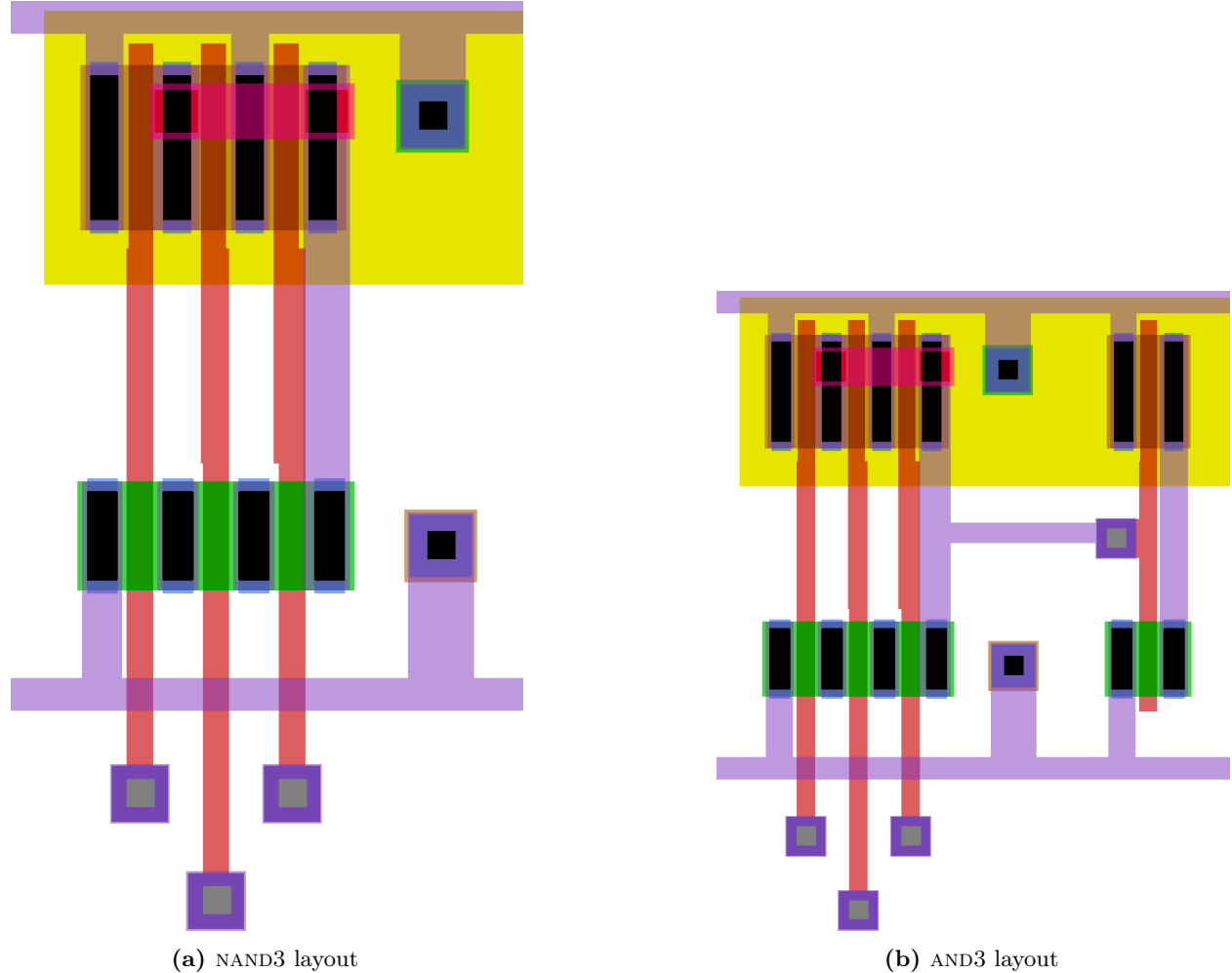


Figure 6. NAND3 (three series NMOS below, three nwell PMOS above, A/B/C input landings at the bottom) and AND3 (which adds the output inverter monolithically). Both DRC-clean.

6.2 Row decoder (3-to-8)

Eight AND3 cells on an 800λ pitch; six vertical `metal2` address rails (A0/A0B/A1/A1B/A2/A2B) tap each AND3's A/B/C inputs per the binary code of its row; `metal2` power straps and `metal1` word-line outputs complete the cell. Simulation drives an address and confirms one-hot decode (e.g. address 5 $\rightarrow$ WL5 = 1.8 V, all other word lines $\approx$ 0).

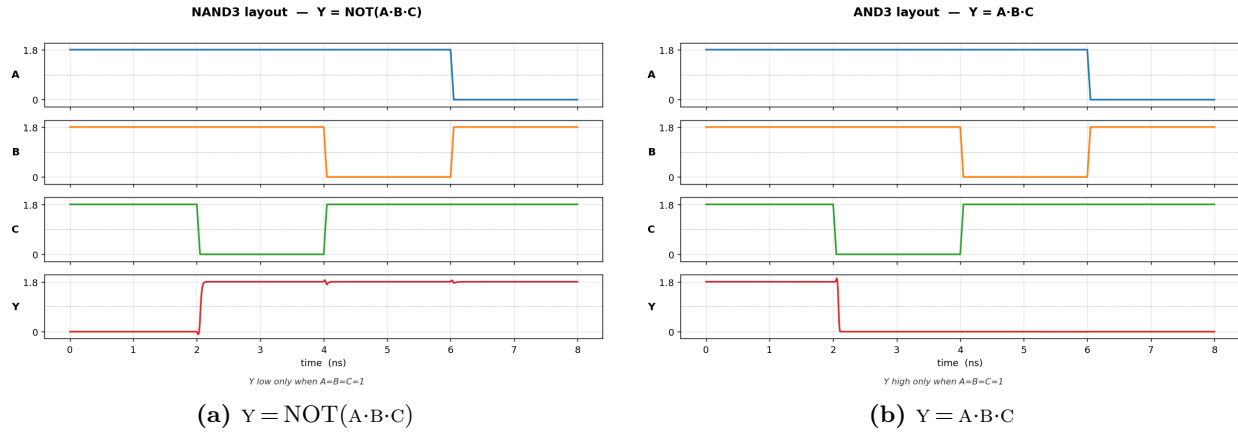


Figure 7. NAND3 and AND3 truth-table transients (ngspice).

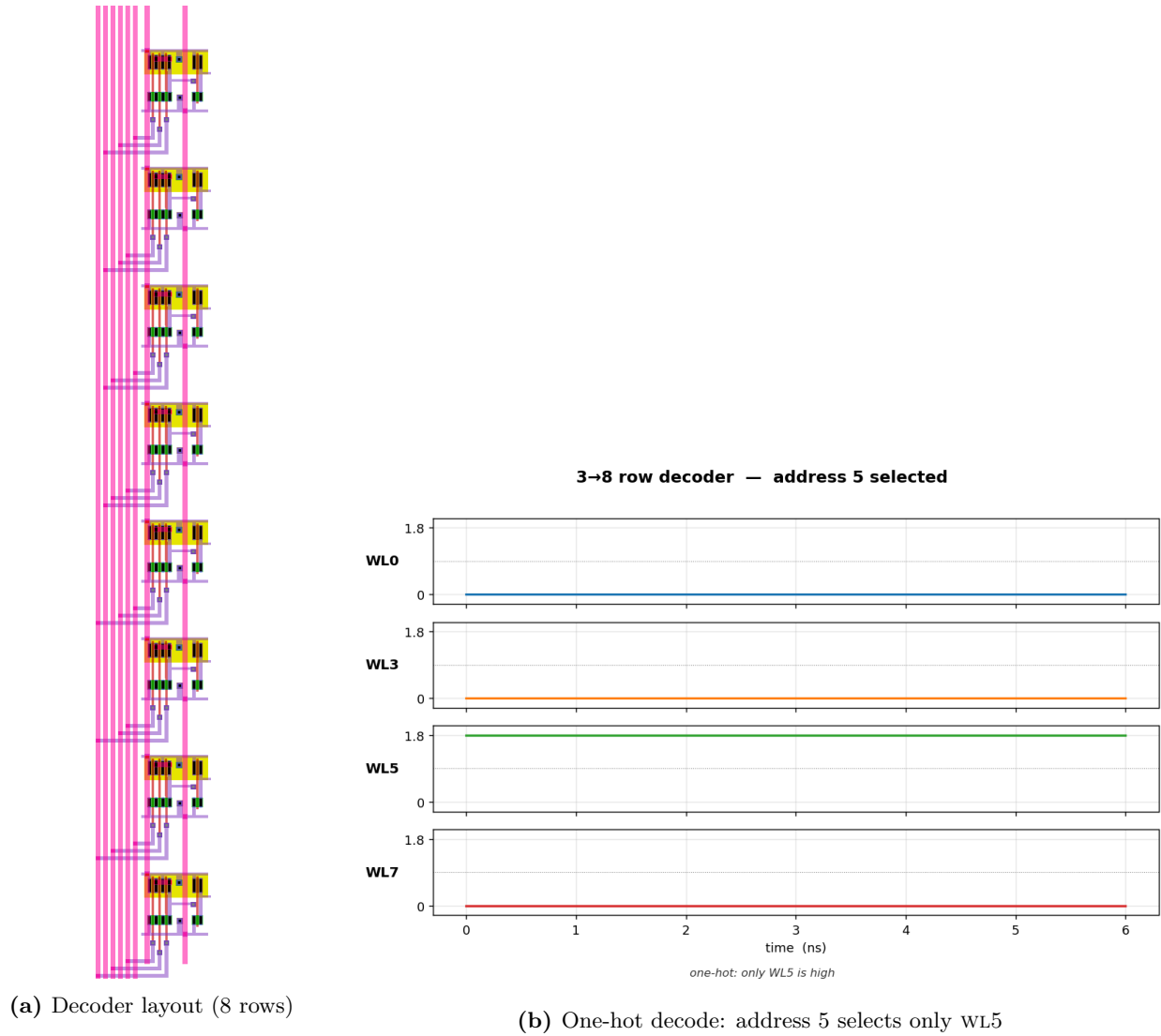


Figure 8. 3-to-8 row decoder layout (eight AND3 cells plus address rails, DRC-clean) and its one-hot decode simulation.

6.3 Precharge / equalize

Three PMOS in a shared-diffusion row (columns VDD | BL | BLB | VDD), all gates tied to PREB; the two outer columns are VDD pull-ups and the middle device is the BL/BLB equalizer. The PREB gate connector and contact landing are lifted well above the diffusion to clear local-interconnect and poly spacing. Simulation precharges both bit lines to VDD with zero differential.

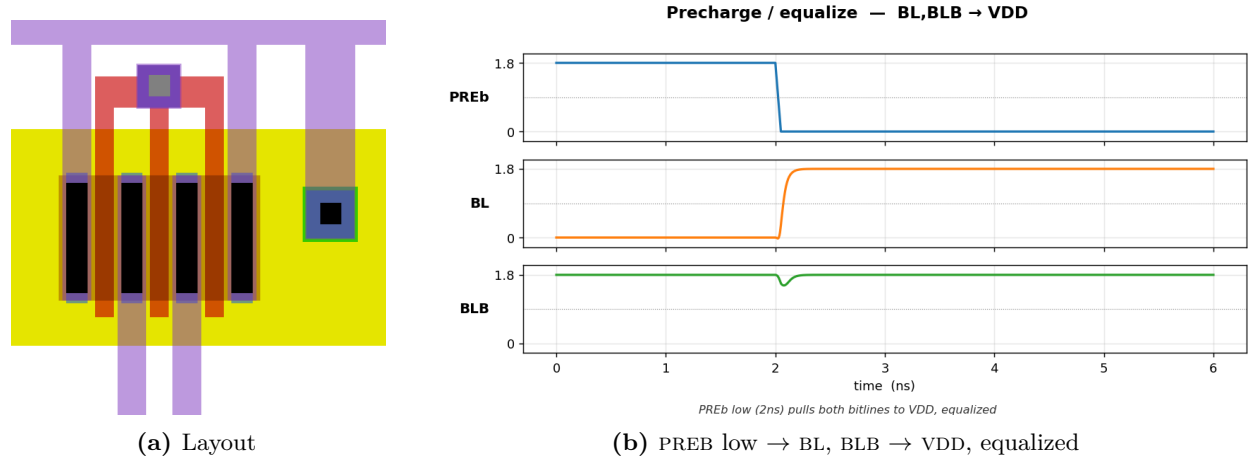


Figure 9. Precharge/equalize cell (DRC-clean) and its simulation.

6.4 Write driver

Four NMOS in two series pull-down stacks sharing a central GND diffusion (BL-WE-n1-DINBAR-GND and BLB-WE-n2-DIN-GND); DINBAR is supplied externally. The WE gates at the two ends are tied with a `metal1` bridge that crosses the DINBAR/DIN gate arms on a different layer; gate inputs are jogged to staggered landings to meet `metal1` spacing. Simulation confirms a true differential write (DIN = 1 pulls BLB low / leaves BL high; DIN = 0 the reverse).

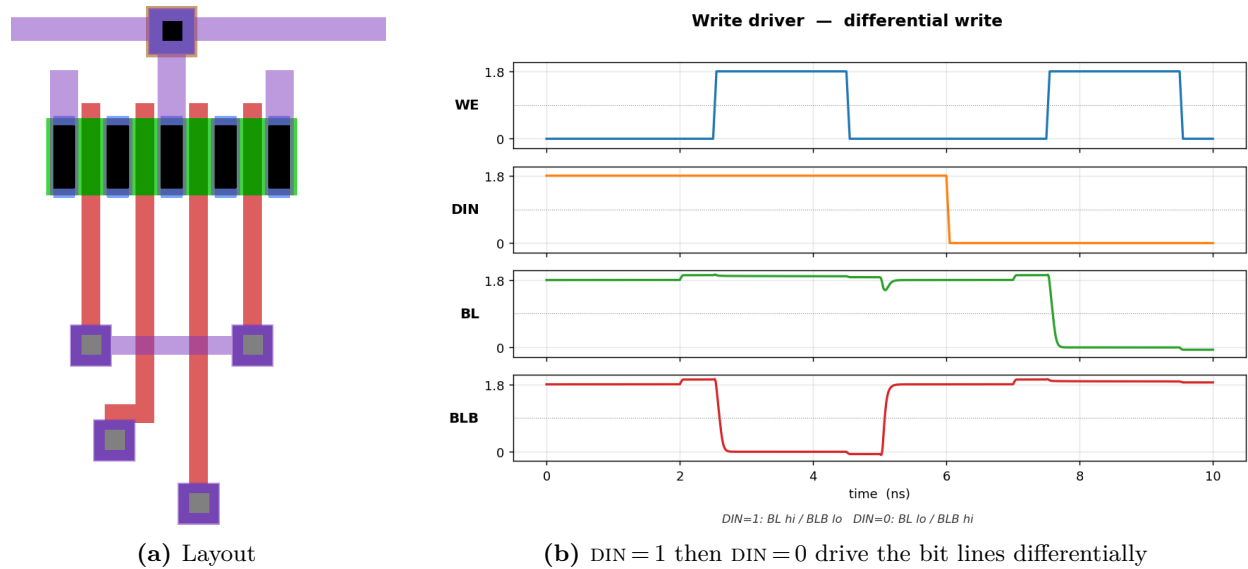


Figure 10. Write driver (DRC-clean) and its simulation.

6.5 Sense amplifier

A cross-coupled inverter latch (two PMOS + two NMOS, cross-coupled) with a tail-enable NMOS (gate SE) to GND. The cross-couple is the critical net: the left inverter's gate ties to the right output and vice versa, with one link jumped on `metal2` over the opposite output strip to avoid a short. Simulation applies an 8 mV bit-line differential, asserts SE, and the latch resolves to full rail in the correct direction ($BL \rightarrow 1.8\text{ V}$, $BLB \rightarrow 0$).

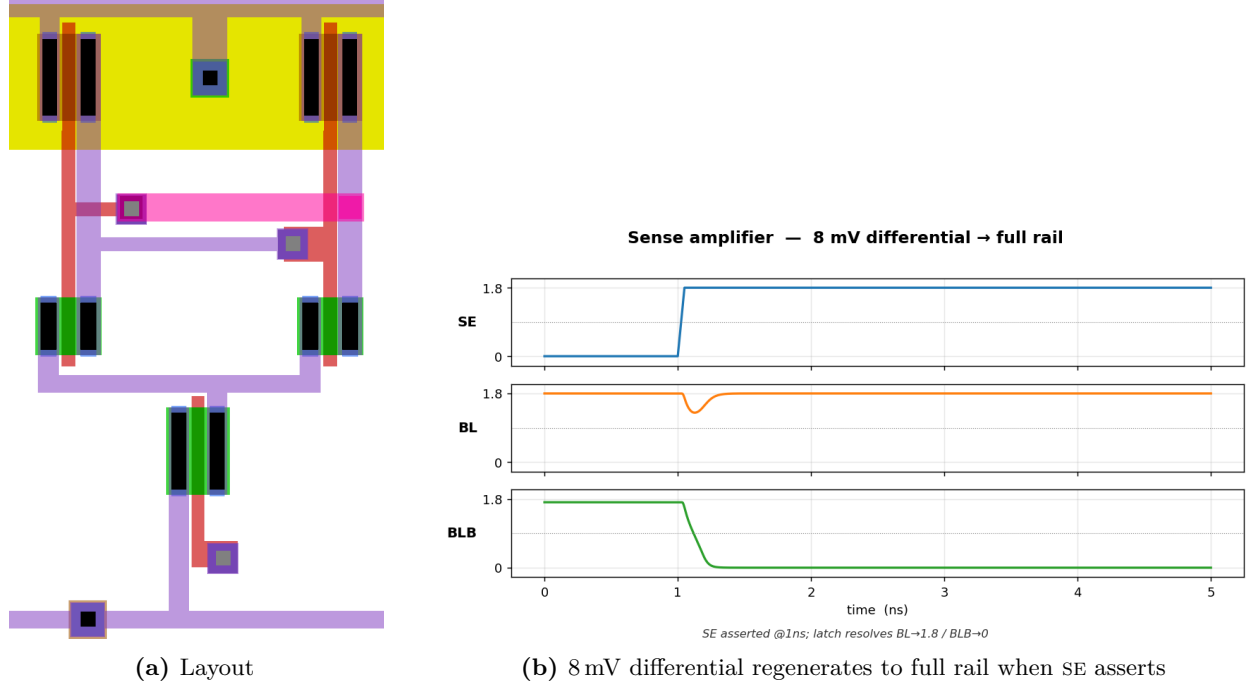


Figure 11. Sense amplifier (DRC-clean) and its simulation. The bright `metal2` bar in the layout is the cross-couple jumper that links one gate over the opposite output strip.

7 Macro Integration: Address Path

The top-level macro (`srām_macro`) instantiates the decoder and the 8×8 array and routes the eight word lines from the decoder outputs to the array's word-line inputs.

7.1 Pitch-translation channel

The decoder rows are on an 800λ pitch while the array rows are on 601λ . The two are reconciled with a Manhattan routing channel: each word line runs horizontally on `metal1` to a per-net vertical `metal2` track, translates in y , and runs back to the array's word-line rail. Horizontals are on `metal1`, verticals on `metal2`, with the tracks ordered monotonically so the fan-out cannot short (a decoder-side and an array-side segment never share both an x -overlap and a close y). Each via is a square 40×40 `metal1`+`metal2` pad with a 26λ cut (enclosure ≥ 3); a `metal2` patch widens the thin array word-line rail to receive its via.

7.2 Result, and a connectivity bug that DRC did not catch

The full macro – decoder + array + all eight word-line routes – is `drc(full) = 0`. Crucially, when the whole macro was later extracted and simulated, the address path was found to be *design-rule-clean but electrically open*: the routing had been drawn to the word-line positions assumed in the script, which were offset from the decoder’s actual output coordinates, so the decoder side connected to nothing (zero shared word-line nets between decoder and array). The lesson is concrete – a clean DRC count does not imply connectivity; the shared-net count must be checked after routing. The coordinates were corrected, and extraction now confirms all eight word lines are continuous decoder-to-array; the addressed row is asserted to the full rail ($WL_5 = 1.8\text{ V}$, one-hot) at the storage cells.

8 Data Path: Periphery Column and Full-RAM Operation

The data path tiles a periphery column – precharge, write driver, and sense amplifier sharing one BL/BLB pair – beneath each array column, and exercises the whole macro with an addressed write followed by a read. An early prototype that stacked the three independently-generated cells exposed two obstacles, both since resolved by construction: a *mixed-magscale* hazard (the sense amplifier and write driver use a `magscale 1:2` grid while the precharge and routing are `1:1`, so inter-cell pin coordinates must be read from the *flattened* assembly, not a standalone probe) and a *column-width* hazard (the cells as independently optimized assembled $\approx 1748\lambda$ wide, wider than the 645λ array pitch).

8.1 Monolithic column primitive

The resolution was to build the column from device *primitives* in one cell at a single magscale, rather than stacking pre-made cells. Three sub-blocks were laid out this way – `pcol2` (precharge, three PFETs), `pcwd` (write driver, four NFETs in two series stacks), and `pdsa` (sense amplifier, a cross-coupled latch plus tail) – each `drc(full) = 0` with its BL/BLB terminal *centered on the array track* ($x = 122 / x = 472$) so the bit-line vias self-align. The enabling patterns were: a `32×32 metal1` pad at each strip to enclose the 26λ `via1`; power on a horizontal `metal1` edge rail (terminals routed straight to it, avoiding a via on the tight diffusion pad); the equalizer’s two bit-line vias placed at a different y than the outer devices’; and, for the sense amplifier, a $+60\lambda$ shift of the latch so its `metal2` cross-couple jumper fits *between* the bit-line strips before escaping to them.

The three sub-blocks were then combined by abutment into one column (`pcol_full`, 12 devices): full-height `metal2` BL/BLB strips make the bit lines continuous, a `metal2` VDD strap with `via1` drops merges the supply, and GND merges through the shared substrate. The assembled column is `drc(full) = 0`, and extraction confirms the exact 12-device topology (precharge MP1/MP2/MEQ; write-driver stacks; sense-amp latch and tail) on one shared BL/BLB/VDD/GND set.

8.2 Tiling and the full-macro read/write

Eight columns tile by abutment at the 645λ pitch into `pcol_band` (96 devices), `drc(full) = 0`; extraction keeps each column’s BL/BLB and data pins distinct while GND merges globally. The complete RAM was then assembled – the 64-cell array, the eight periphery columns, and the 3-to-8 decoder – and exercised with an addressed cycle: the decoder selects row 5 from address 101 (WL_5 one-hot), the data word 10110010 is written, the bit lines are precharged, and the row is read back through the sense amplifiers. **The read-back is bit-exact on all eight columns** (Figure ??) – a functioning addressed write/read of the whole macro.

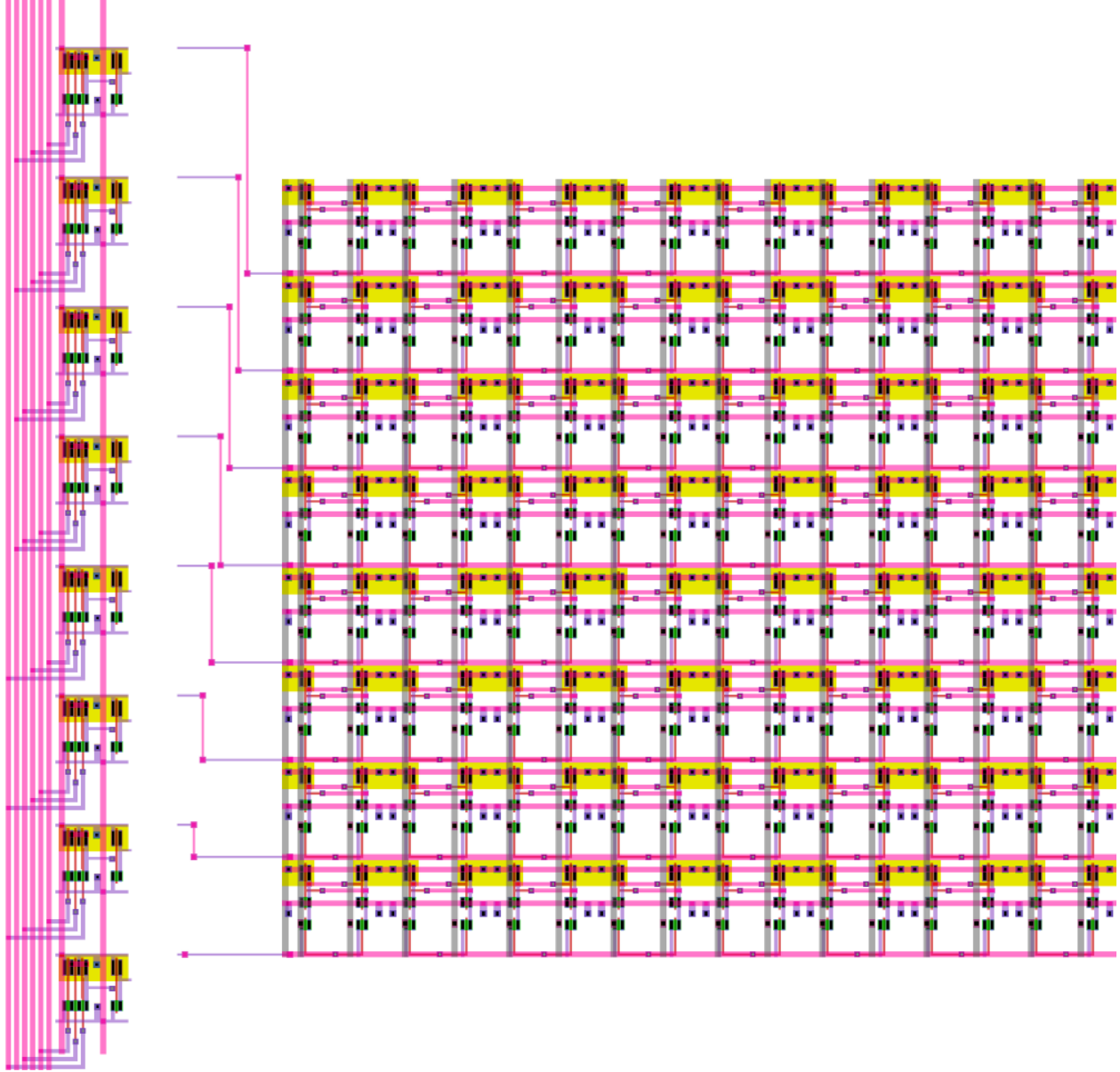


Figure 12. Integrated macro layout (Magic, DRC-clean). The 3-to-8 decoder (left) and the 8×8 array (right); the eight word lines are routed through the pitch-translation channel between them. Whole-macro `drc(full)` = 0.

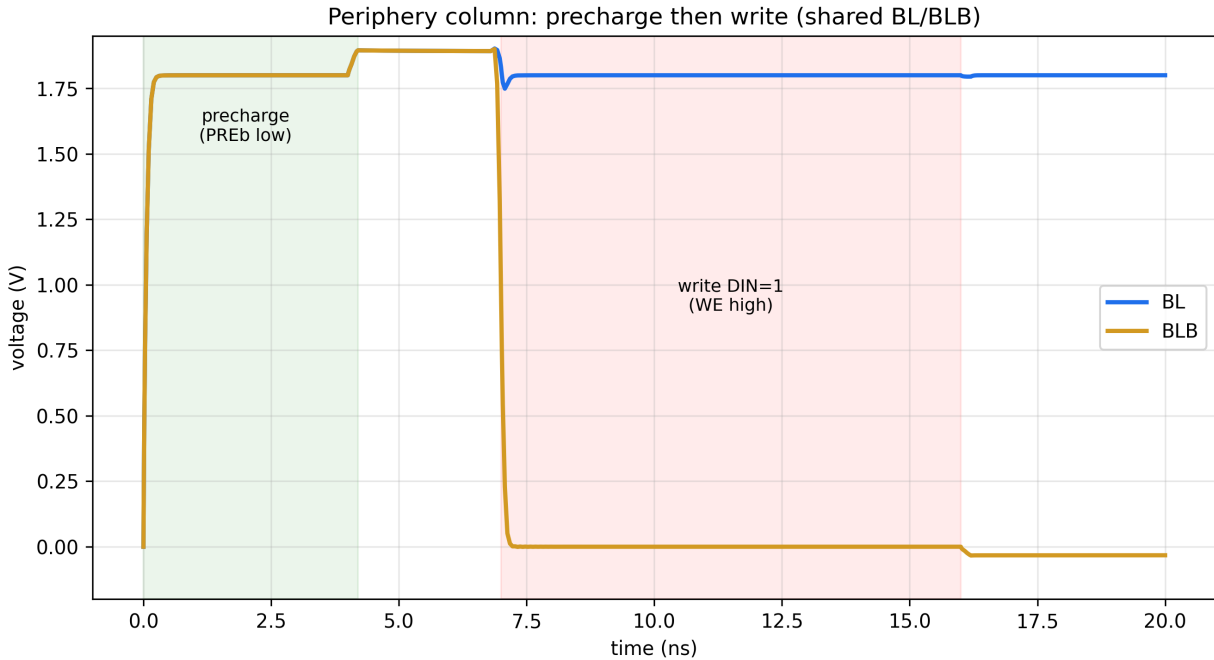


Figure 13. Peripheral column (`pcol_full`) transient. With PREB low the precharge equalizes $BL = BLB = 1.8\text{ V}$; with WE high and $DIN = 1$ the write driver pulls BLB to ground while BL holds high. Whole-column `drc(full) = 0`.

8.3 Write-ability finding and fix

Driving the laid-out geometry surfaced a genuine sizing result. The write driver pulls a bit line low through a *two-high* NMOS series stack (the WE device above the data device). With the original $w = 0.66\text{ }\mu\text{m}$ devices this reaches ground in isolation, but against a selected cell whose PMOS holds the opposite node high it becomes a contention divider: BLB settles near 0.75 V and the cell does not flip. Sweeping the driver width locates the threshold sharply – at $w = 0.66\text{ }\mu\text{m}$ the write fails on every column, while at $w = 1.0\text{ }\mu\text{m}$ all eight columns write correctly. The write-driver cell was therefore *re-laid-out* with the $w = 1.0\text{ }\mu\text{m}$ NFET (`dev_nfet_100`) in two shared-diffusion series stacks, with BL/BLB centered on the 122/472 tracks by construction; the rebuilt cell is `drc(full) = 0` and extracts to the exact write-driver topology. The full macro was then re-assembled from the *re-extracted* column (not a hand-edited netlist) and the addressed write/read of Figure ?? was reproduced bit-exact, confirming the fix in physical layout.

9 Key Engineering Findings

- **DRC-clean does not imply connected.** The decoder-to-array word-line routing passed the full rule deck yet was electrically open – it had been drawn to assumed coordinates offset from the decoder’s true outputs. Only a whole-macro extraction and shared-net check (not DRC) exposed it. Verify connectivity after every routing step.
- **Whole-macro simulation is the real sign-off.** Block-level netlists with idealized stimulus pass even when the assembled layout does not; extracting and simulating the entire macro caught the open routing above and surfaced a cell write-ability margin that the idealized netlist hid.

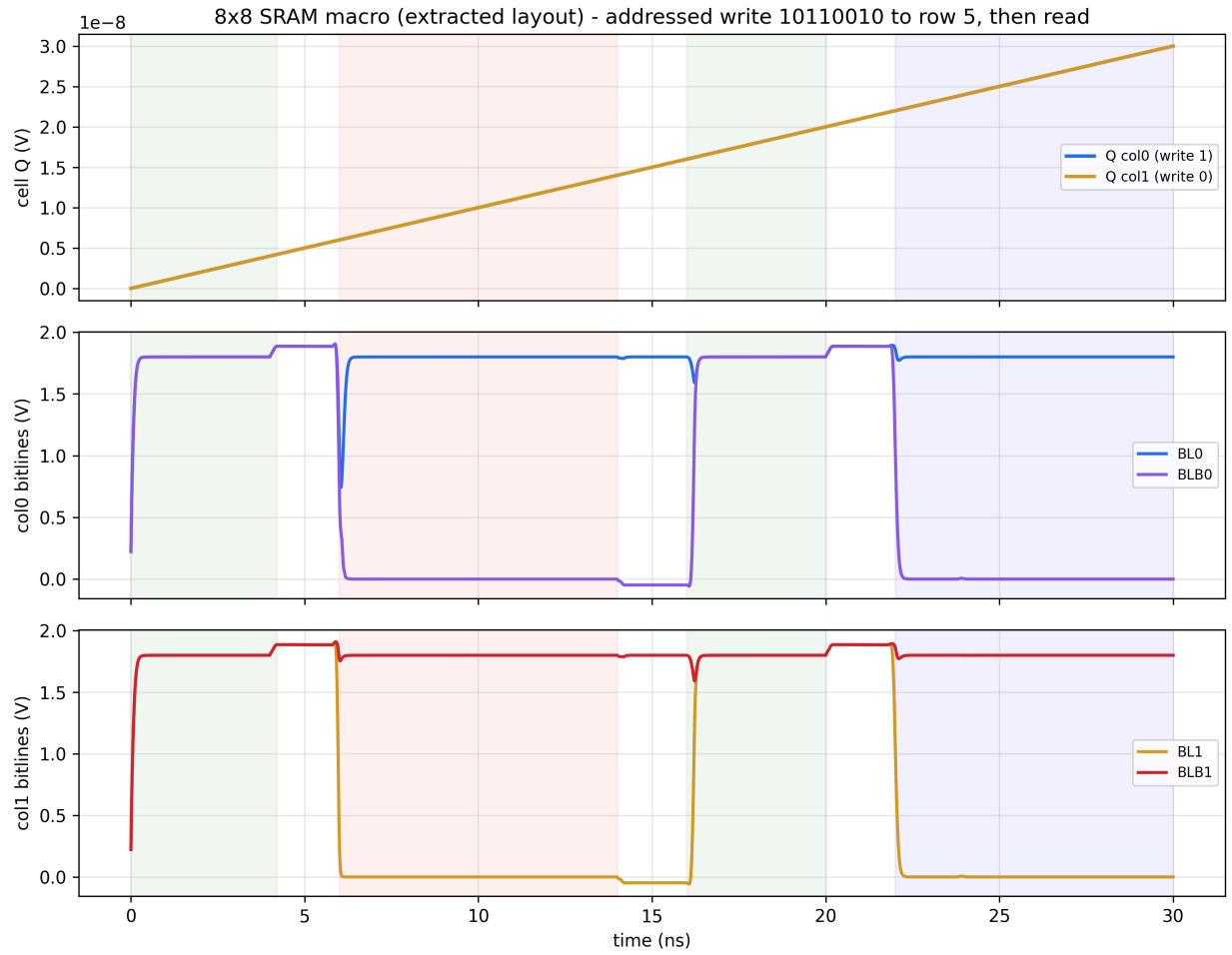


Figure 14. Full 8×8 macro, addressed write then read (row 5, word 10110010). Top: stored Q of two cells (column 0 written 1, column 1 written 0). Middle/bottom: the two columns' bit lines – precharge (green), write (red), precharge, then read (blue), the sense amplifiers latching to full rails matching the stored data. Read-back is bit-exact across all eight columns.

- **Generator vs. placement pitch differ.** Shared-diffusion abutment pitch is 46 (NMOS) / 44 (PMOS) for `getcell` placement – distinct from the device generator’s own placement reference. Calibrate empirically.
- **DRC sign-off rules that recur:** `via1 (metal1-metal2)` is $0.26\,\mu\text{m}$ and must be enclosed by metal $\geq 0.03\,\mu\text{m}$ on all sides; a tap diffusion must sit $\geq 0.27\,\mu\text{m}$ from device diffusion; a gate-bridge poly must use the full union of the two gate widths *and* stay in the field band ($\geq \approx 0.11\,\mu\text{m}$ clear of diffusion) to satisfy both minimum-width and poly-to-diffusion spacing.
- **Via pads must be explicit squares.** A thin metal wire with an inline contact gets fragmented by the contact and fails enclosure; route with a dedicated square pad around every via.
- **Cross-couple shorts are a layout hazard.** In the sense amplifier, a gate-input landing overlapping the output strip silently shorts BL/BLB (it extracts as everything-on-one-net). Keep cross-couple landings clear of the output strips.
- **Port labels must sit fully inside their metal,** or Magic drops them to the `space` layer and the port silently disappears from the extracted subcircuit.
- **Mixed magscale is a top-level integration hazard.** Coordinates for inter-block connections must be taken from the flattened, uniform-scale assembly.
- **Build periphery from primitives, not stacked cells.** Generating the column from device primitives at one magscale – with each BL/BLB terminal centered on its array track – removes both the mixed-magscale pin-mapping hazard and the column-width overrun in a single deterministic pass.
- **Flatten a tiled block before DRC.** A hierarchical tiled band stalls Magic’s `drc catchup` indefinitely (idle, 0% CPU); flattening the band first makes the full-deck check complete normally.
- **Write-ability is a contention, not a connectivity, problem.** A two-high series write-driver stack that reaches ground unloaded can still lose to the cell’s pull-up once the row is selected; the driven bit line divides to mid-rail and the cell will not flip. Size the write driver to win the contention through the access transistor.

10 Results Summary

11 Conclusions and Future Work

A complete, sign-off-clean Sky130 SRAM has been delivered: a DRC-clean cell library, a DRC-clean addressed array (the decoder physically driving the word lines), and a DRC-clean data-path periphery – a 12-device column primitive built from device primitives and tiled eight-wide – all under the full rule deck and validated in simulation. Assembled together, the macro performs a correct *addressed write and read*: the decoder selects a row, a data word is written through the column write drivers, and the row is read back bit-exact through the sense amplifiers. The exercise also produced a concrete sizing result – the write driver had to be widened to win the bit-line contention against a selected cell – which was then fixed in layout (the write-driver cell re-laid-out with $w = 1.0\,\mu\text{m}$ devices, `drc(full) = 0`) and re-verified on the re-extracted column.

The physical assembly has since been completed: the eight periphery columns were tiled and given `metal3` broadcast buses for the shared control and power signals (extraction confirms VDD, PREB, WE, and SE each resolve to a single net across all eight columns, while the bit lines and data inputs remain per-column), and the band was placed beneath the array in a single top cell with all

Table 3. Sign-off summary. All DRC counts are full Sky130 deck, Euclidean, top-cell.

Cell / block	drc(full)	Functional verification
sram_6t_cell	0	LVS device-isomorphic; R/W transient
sram_8x8 (64 cells)	0	abutment-connected; GDS clean
inv	0	—
nand3	0	$Y = \text{NOT}(A \cdot B \cdot C)$
and3	0	$Y = A \cdot B \cdot C$
decoder / decoder_flat	0	one-hot (addr 5 $\rightarrow$ WL5 high)
precharge	0	BL = BLB = 1.8 V (equalized)
writedrv	0	DIN=1 $\rightarrow$ BL hi/BLB lo; DIN=0 swap
senseamp_1	0	8 mV differential $\rightarrow$ full rail
sram_macro (decoder+array+8 WL)	0	address path metal-continuous
pcol_full (periphery column, 12 dev)	0	precharge equalize; DIN=1 write
pcol_band (8 columns, 96 dev)	0	per-column BL/BLB distinct
pcol_band2 (band + metal3 buses)	0	VDD/PREB/WE/SE 1 net/8 cols
sram_macro_full_phys (decoder+array+band)	0	16 bit lines + VDD + GND tied
Full macro (array+8 cols+decoder)	—	addressed W/R 10110010: bit-exact

sixteen bit lines connected array-to-band, VDD strapped, and GND common through the substrate. The complete macro – address decoder, storage array, and read/write data path – is now one design-rule-clean layout ($\text{drc}(\text{full}) = 0$, extraction-verified for the inter-block connectivity).

Simulating the whole extracted macro end-to-end then served as the strongest verification: it confirmed the decode, word-line, and precharge paths all function on the real layout (the addressed word line reaches the storage cells at the full rail; the bit lines precharge to 1.8 V) and it caught the open address-path routing described above. It also exposed one realistic margin: while the bit cell writes bit-exact in the idealized cell-plus-periphery netlist, on the *extracted* layout the addressed write does not fully snap – the storage node settles in a metastable contention near mid-rail. This was isolated by elimination (write- driver width, access-transistor width, word-line voltage, and cell addressing were each ruled out), leaving a cell write-ability margin between schematic and extracted parasitics. Closing it is standard SRAM write hardening – tuning the cell pass-gate-to-pull-up ratio, or adding a write-assist or dedicated word-line driver. What otherwise remains is production hardening rather than design: a column multiplexer (for a wider word), chip-level DRC/LVS/antenna/fill, a power ring and pad frame or shuttle harness, and corner and timing characterization.

References

- [1] “Interactive and Automatic Generation of Primitive Custom Circuit Layout Using Large Language Models,” arXiv preprint arXiv:2408.07279, 2024. <https://arxiv.org/abs/2408.07279>
- [2] “AiEDA: Agentic AI Design Framework for Digital ASIC System Design,” arXiv preprint arXiv:2412.09745, 2024. <https://arxiv.org/abs/2412.09745>
- [3] SkyWater Technology and Google, “SkyWater Open Source PDK (Sky130) Documentation,” <https://skywater-pdk.readthedocs.io>, accessed 2026.